welcome

Version: v1.0

Date: 2026-01-01

1. API overview (v1.0)

API overview (v1.0)

v1.0 snapshot of the same API as **Latest**. Use the version dropdown to compare.

Base URL

```
https://api.example.com/v1
```

Authentication

```
Authorization: Bearer <access_token>
```

Endpoint list

List users

Request method

`GET`

Request URL

`https://api.example.com/v1/users`
Query: `page` (default `1`), `pageSize` (default `20`).

Request body

None.

Response fields (`200 OK`)

- `items`, `page`, `pageSize`, `total` — same shape as Latest.

Request example

```
GET /v1/users?page=1&pageSize=20 HTTP/1.1
Host: api.example.com
Authorization: Bearer <access_token>
Accept: application/json
```

Response example

```
{
  "items": [
```

```
    { "id": "usr_001", "email": "alice@example.com" },
    { "id": "usr_002", "email": "bob@example.com" }
  ],
  "page": 1,
  "pageSize": 20,
  "total": 2
}
```

Get current user

Request method

`GET`

Request URL

`https://api.example.com/v1/users/me`

Request body

None.

Response fields (`200 OK`)

- `id`, `email`

Request example

```
GET /v1/users/me HTTP/1.1
Host: api.example.com
Authorization: Bearer <access_token>
Accept: application/json
```

Response example

```
{
  "id": "usr_001",
  "email": "alice@example.com"
}
```

Create user

Request method

`POST`

Request URL

`https://api.example.com/v1/users`

Request body (JSON)

- `email` (required), `displayName` (optional)

Response fields (`201 Created`)

- `id`, `email`, `displayName`

Request example

```
POST /v1/users HTTP/1.1
Host: api.example.com
Authorization: Bearer <access_token>
Content-Type: application/json
Accept: application/json

{"email":"carol@example.com","displayName":"Carol"}
```

Response example

```
{
  "id": "usr_003",
  "email": "carol@example.com",
  "displayName": "Carol"
}
```

See also

Latest → `API` → `API overview`.

2. FAQ

FAQ

Why does GitHub Actions show Node.js deprecation warnings?

Some GitHub-maintained Pages actions still reference older runtimes internally. If your workflow is green, deployment is still successful.

How do I preview my doc changes?

Run:

```
npm run start
```

Then open `http://localhost:3000`.

How do I verify before pushing?

Run:

```
npm run build
```

This catches broken links and build issues early.

3. Getting Started

Getting Started

Use this page as a quick checklist to run the docs site locally.

Prerequisites

- Node.js 20 or later
- npm 10 or later

Install dependencies

```
npm ci
```

Start local development

```
npm run start
```

The site will be available at `http://localhost:3000`.

Build for production

```
npm run build
```

If build succeeds, your content is ready to deploy.

4. Installation Guide

Installation Guide

This guide explains a clean setup flow for contributors.

1. Clone the repository

```
git clone https://github.com/MonicaChenc/Monica-test.git
cd Monica-test
```

2. Install dependencies

```
npm ci
```

3. Run the docs site

```
npm run start
```

4. Verify production build

```
npm run build
```

If the build command passes, your environment is configured correctly.

5. Welcome

Welcome

Welcome to the Monica Docs site.

This documentation is organized to help you get productive quickly:

- Start with the basics in **Getting Started**
- Follow practical steps in **Guides**
- Check common questions in **FAQ**

How to use this documentation

If you are new here, read these pages in order:

1. `Getting Started`
2. `Installation Guide`
3. `FAQ`

If you already know the basics, jump directly to the guide you need.

Contributing

Open a pull request with your doc updates and include a short summary of what changed.

6. Congratulations!

Congratulations!

You have just learned the **basics of Docusaurus** and made some changes to the **initial template**.

Docusaurus has **much more to offer**!

Have **5 more minutes**? Take a look at **versioning** and **i18n**.

Anything **unclear** or **buggy** in this tutorial? Please report it!

What's next?

- Read the official documentation
- Modify your site configuration with `docusaurus.config.js`
- Add navbar and footer items with `themeConfig`
- Add a custom Design and Layout
- Add a search bar
- Find inspirations in the Docusaurus showcase
- Get involved in the Docusaurus Community

7. Create a Blog Post

Create a Blog Post

Docusaurus creates a **page for each blog post**, but also a **blog index page**, a **tag system**, an **RSS** feed...

Create your first Post

Create a file at `blog/2021-02-28-greetings.md`:

```
---
slug: greetings
title: Greetings!
authors:
  - name: Joel Marcey
    title: Co-creator of Docusaurus 1
    url: https://github.com/JoelMarcey
    image_url: https://github.com/JoelMarcey.png
  - name: Sébastien Lorber
    title: Docusaurus maintainer
    url: https://sebastienlorber.com
    image_url: https://github.com/slorber.png
tags: [greetings]
---

Congratulations, you have made your first post!

Feel free to play around and edit this post as much as you like.
```

A new blog post is now available at http://localhost:3000/blog/greetings.

8. Create a Document

Create a Document

Documents are **groups of pages** connected through:

- a **sidebar**
- **previous/next navigation**
- **versioning**

Create your first Doc

Create a Markdown file at `docs/hello.md`:

```
# Hello

This is my **first Docusaurus document**!
```

A new document is now available at http://localhost:3000/docs/hello.

Configure the Sidebar

Docusaurus automatically **creates a sidebar** from the `docs` folder.

Add metadata to customize the sidebar label and position:

```
---
sidebar_label: 'Hi!'
sidebar_position: 3
---

# Hello

This is my **first Docusaurus document**!
```

It is also possible to create your sidebar explicitly in `sidebars.js`:

```
export default {
  tutorialSidebar: [
    'intro',
    // highlight-next-line
    'hello',
    {
      type: 'category',
```

```
      label: 'Tutorial',
      items: ['tutorial-basics/create-a-document'],
    },
  ],
};
```

9. Create a Page

Create a Page

Add **Markdown or React** files to `src/pages` to create a **standalone page**:

- `src/pages/index.js` → `localhost:3000/`
- `src/pages/foo.md` → `localhost:3000/foo`
- `src/pages/foo/bar.js` → `localhost:3000/foo/bar`

Create your first React Page

Create a file at `src/pages/my-react-page.js`:

```
import React from 'react';
import Layout from '@theme/Layout';

export default function MyReactPage() {
  return (
    <Layout>
      <h1>My React page</h1>
      <p>This is a React page</p>
    </Layout>
  );
}
```

A new page is now available at http://localhost:3000/my-react-page.

Create your first Markdown Page

Create a file at `src/pages/my-markdown-page.md`:

```
# My Markdown page

This is a Markdown page
```

A new page is now available at http://localhost:3000/my-markdown-page.

10. Deploy your site

Deploy your site

Docusaurus is a **static-site-generator** (also called **Jamstack**).

It builds your site as simple **static HTML, JavaScript and CSS files**.

Build your site

Build your site **for production**:

```
npm run build
```

The static files are generated in the `build` folder.

Deploy your site

Test your production build locally:

```
npm run serve
```

The `build` folder is now served at http://localhost:3000/.

You can now deploy the `build` folder **almost anywhere** easily, **for free** or very small cost (read the **Deployment Guide**).

11. Markdown Features

Markdown Features

Docusaurus supports **Markdown** and a few **additional features**.

Front Matter

Markdown documents have metadata at the top called Front Matter:

```
// highlight-start
---
id: my-doc-id
title: My document title
description: My document description
slug: /my-custom-url
---
// highlight-end

Markdown content
```

Headings {/* #my-heading-id */}

Markdown headings are supported using the standard "#" syntax and are automatically added to the table of contents. The number of # corresponds to the heading level.

```
## Headings

My text
```

Heading Ids {/* #my-custom-id */}

Add `{/* #my-custom-id */}` after the heading text to assign it an explicit anchor id, used for linking.

```
### Heading Ids {/_ #my-custom-id _/}
```

Links

Regular Markdown links are supported, using url paths or relative file paths.

```
Let's see how to [Create a page](/create-a-page).
```

```
Let's see how to [Create a page](./create-a-page.mdx).
```

Result: Let's see how to Create a page.

Images

Regular Markdown images are supported.

You can use absolute paths to reference images in the static directory (`static/img/docusaurus.png`):

```
![Docusaurus logo](/img/docusaurus.png)
```

Docusaurus logo

You can reference images relative to the current file as well. This is particularly useful to colocate images close to the Markdown files using them:

```
![Docusaurus logo](./img/docusaurus.png)
```

Code Blocks

Markdown code blocks are supported with Syntax highlighting.

```
```jsx title="src/components/HelloDocusaurus.js"
function HelloDocusaurus() {
  return <h1>Hello, Docusaurus!</h1>;
}
```
```

```
function HelloDocusaurus() {
  return <h1>Hello, Docusaurus!</h1>;
}
```

Admonitions

Docusaurus has a special syntax to create admonitions and callouts:

```
:::tip[My tip]

Use this awesome feature option

:::

:::danger[Take care]
```

```
This action is dangerous

:::
```

:::tip[My tip]

Use this awesome feature option

:::

:::danger[Take care]

This action is dangerous

:::

MDX and React Components

MDX can make your documentation more **interactive** and allows using any **React components inside Markdown**:

```
export const Highlight = ({children, color}) => (
  <span
    style={{
      backgroundColor: color,
      borderRadius: '20px',
      color: '#fff',
      padding: '10px',
      cursor: 'pointer',
    }}
    onClick={() => {
      alert(`You clicked the color ${color} with label ${children}`)
    }}>
    {children}
  </span>
);

This is <Highlight color="#25c2a0">Docusaurus green</Highlight> !

This is <Highlight color="#1877F2">Facebook blue</Highlight> !
```

export const Highlight = ({children, color}) => (<span style={{ backgroundColor: color, borderRadius: '20px', color: '#fff', padding: '10px', cursor: 'pointer', }} onClick={() => { alert(`You clicked the color ${color} with label ${children}`); }}> {children});

This is Docusaurus green !

This is Facebook blue !

12. Manage Docs Versions

Manage Docs Versions

Docusaurus can manage multiple versions of your docs.

Create a docs version

Release a version 1.0 of your project:

```
npm run docusaurus docs:version 1.0
```

The `docs` folder is copied into `versioned_docs/version-1.0` and `versions.json` is created.

Your docs now have 2 versions:

- `1.0` at `http://localhost:3000/docs/` for the version 1.0 docs
- `current` at `http://localhost:3000/docs/next/` for the **upcoming, unreleased docs**

Add a Version Dropdown

To navigate seamlessly across versions, add a version dropdown.

Modify the `docusaurus.config.js` file:

```
export default {
  themeConfig: {
    navbar: {
      items: [
        // highlight-start
        {
          type: 'docsVersionDropdown',
        },
        // highlight-end
      ],
    },
  },
};
```

The docs version dropdown appears in your navbar:

Docs Version Dropdown

Update an existing version

It is possible to edit versioned docs in their respective folder:

- `versioned_docs/version-1.0/hello.md` updates `http://localhost:3000/docs/hello`
- `docs/hello.md` updates `http://localhost:3000/docs/next/hello`

13. Translate your site

Translate your site

Let's translate `docs/intro.md` to French.

Configure i18n

Modify `docusaurus.config.js` to add support for the `fr` locale:

```
export default {
  i18n: {
    defaultLocale: 'en',
    locales: ['en', 'fr'],
  },
};
```

Translate a doc

Copy the `docs/intro.md` file to the `i18n/fr` folder:

```
mkdir -p i18n/fr/docusaurus-plugin-content-docs/current/

cp docs/intro.md i18n/fr/docusaurus-plugin-content-docs/current/intro.md
```

Translate `i18n/fr/docusaurus-plugin-content-docs/current/intro.md` in French.

Start your localized site

Start your site on the French locale:

```
npm run start -- --locale fr
```

Your localized site is accessible at http://localhost:3000/fr/ and the `Getting Started` page is translated.

:::caution

In development, you can only use one locale at a time.

:::

Add a Locale Dropdown

To navigate seamlessly across languages, add a locale dropdown.

Modify the `docusaurus.config.js` file:

```
export default {
  themeConfig: {
    navbar: {
      items: [
        // highlight-start
        {
          type: 'localeDropdown',
        },
        // highlight-end
      ],
    },
  },
};
```

The locale dropdown now appears in your navbar:

Locale Dropdown

Build your localized site

Build your site for a specific locale:

```
npm run build -- --locale fr
```

Or build your site to include all the locales at once:

```
npm run build
```